

S10/L5

Traccia:

Con riferimento al file Malware_U3_W2_L5 presente all'interno della cartella «Esercizio_Pratico_U3_W2_L5» sul desktop della macchina virtuale dedicata per l'analisi dei malware, rispondere ai seguenti quesiti:

1. Quali librerie vengono importate dal file eseguibile? Fare anche una descrizione
2. Quali sono le sezioni di cui si compone il file eseguibile del malware? Fare anche una descrizione

Con riferimento alla figura nella slide successiva, risponde ai seguenti quesiti:

3. Identificare i costrutti noti (creazione dello stack, eventuali cicli, altri costrutti)
4. Ipotesizzare il comportamento della funzionalità implementata
5. Fare una tabella per spiegare il significato delle singole righe di codice

```
push    ebp
mov     ebp, esp
push    ecx
push    0          ; dwReserved
push    0          ; lpdwFlags
call    ds:InternetGetConnectedState
mov     [ebp+var_4], eax
cmp     [ebp+var_4], 0
jz      short loc_40102B
```

```
push    offset aSuccessInterne ; "Success: Internet Connection\n"
call    sub_40117F
add     esp, 4
mov     eax, 1
jmp     short loc_40103A
```

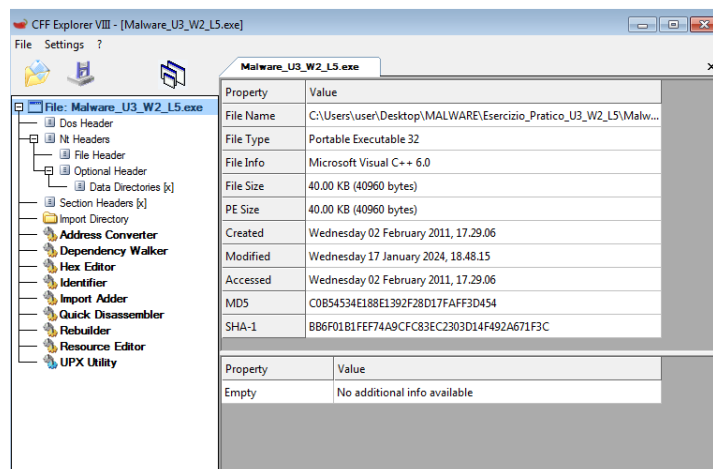
```
loc_40102B:          ; "Error 1.1: No Internet\n"
push    offset aError1_1NoInte
call    sub_40117F
add     esp, 4
xor     eax, eax
```

```
loc_40103A:
mov     esp, ebp
pop     ebp
retn
sub_401000 endp
```

1) Librerie Importate

- Ricerca librerie

Per identificare le librerie usate dal malware possiamo usare l'analisi statica basica. Per farlo usiamo il programma **CFF Explorer** e carichiamo il malware da analizzare.



Dall'elenco presente sulla sinistra del software selezioniamo la voce **“Import Directory”**.

Nella schermata principale appariranno le librerie usate dal malware.

Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
000065EC	N/A	000064DC	000064E0	000064E4	000064E8	000064EC
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	44	00006518	00000000	00000000	000065EC	00006000
WININET.dll	5	000065CC	00000000	00000000	00006664	000060B4

Come possiamo vedere dall'immagine soprastante vengono caricate le librerie **Kernel32** e **Wininet**. Queste librerie sono componenti fondamentali del sistema operativo Windows, ognuna con funzioni specifiche per la gestione delle risorse di sistema e delle comunicazioni di rete. Prima di descrivere le librerie nel dettaglio, c'è da notare che analizzando le funzioni importate dalla libreria Kernel32 troviamo le funzioni **GetProcAddress** e **LoadLibrary**. Queste funzioni permettono il caricamento dinamico delle librerie, quindi a tempo di esecuzione (**run time**). Questo significa che il malware potrebbe caricare altre librerie che non possiamo vedere con una semplice analisi statica basica.

- **Descrizione delle librerie**

Kernel32.dll

Kernel32.dll è una delle librerie di sistema più importanti di Windows. Essa contiene funzioni di base per la gestione delle risorse del sistema operativo e sono utilizzate da quasi tutti i programmi Windows.

Alcune delle sue principali caratteristiche sono:

- Gestione della memoria:
Allocazione e de allocazione della memoria;
Gestione della memoria virtuale;
Funzioni per la copia e lo spostamento di blocchi di memoria.
- Gestione dei processi e dei thread:
Creazione e terminazione di processi e thread;
Sincronizzazione dei thread tramite.
- Gestione delle operazioni di I/O:
Funzioni per leggere e scrivere su file;
Gestione di dispositivi di input/output;
Funzioni per la gestione delle pipe.

Wininet.dll

Wininet.dll è una libreria di Windows che fornisce un'interfaccia per le applicazioni che necessitano di comunicazioni su rete utilizzando protocolli Internet. Questa libreria è progettata per semplificare le operazioni di rete per gli sviluppatori di applicazioni, fornendo un'astrazione per molte delle complessità associate alla gestione delle comunicazioni di rete. Le principali funzioni di Wininet.dll includono:

- Gestione delle connessioni Internet:
Creazione e gestione delle connessioni a server web.
Supporto per connessioni sicure tramite HTTPS.
- Gestione dei protocolli:
Supporto per i protocolli HTTP e FTP.
Funzioni per inviare richieste e ricevere risposte tramite HTTP.
Funzioni per il download e l'upload di file tramite FTP.
- Gestione della cache di Internet:
Funzioni per la gestione della cache del browser.
Operazioni per aggiungere, rimuovere e aggiornare elementi nella cache.
- Gestione delle sessioni di rete:
Funzioni per mantenere sessioni di rete persistenti.
Gestione dei cookie e delle intestazioni http.
- Autenticazione e sicurezza:
Funzioni per gestire l'autenticazione degli utenti.
Supporto per la gestione dei certificati di sicurezza e delle connessioni SSL/TLS.

2) Sezioni del Malware

- Ricerca sezioni

Per la ricerca delle sezioni del malware continuiamo ad utilizzare il tool CFF explorer.

Questa volta selezionando la voce “**Section Header**” nella schermata principale del software compaiono le sezioni del malware.

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00004A78	00001000	00005000	00001000	00000000	00000000	0000	0000	60000020
.rdata	0000095E	00006000	00001000	00006000	00000000	00000000	0000	0000	40000040
.data	00003F08	00007000	00003000	00007000	00000000	00000000	0000	0000	C0000040

- Descrizione delle sezioni

Nel contesto di un file eseguibile utilizzato nel sistema operativo Windows, le sezioni **.text**, **.rdata** e **.data** sono aree di memoria che contengono differenti tipi di informazioni. Queste sezioni sono cruciali per il funzionamento del programma e contengono rispettivamente codice eseguibile, dati in sola lettura e dati modificabili.

.text

La sezione **.text** (o "text") è la sezione del file eseguibile che contiene il codice eseguibile del programma. Questa sezione è di sola lettura ed è generalmente condivisa tra i processi che eseguono lo stesso programma per ottimizzare l'uso della memoria.

.rdata

La sezione **.rdata** (o "read-only data") contiene dati di sola lettura che vengono utilizzati dal programma. Questi dati non devono essere modificati durante l'esecuzione del programma. Questa sezione è utile per contenere dati costanti come stringhe di testo e tabelle di costanti.

.data

La sezione **.data** è la sezione che contiene i dati modificabili del programma. A differenza delle sezioni **.text** e **.rdata**, i dati contenuti in **.data** possono essere letti e scritti durante l'esecuzione del programma. Questa sezione è utilizzata per variabili globali e statiche che il programma deve poter modificare.

3) Costrutti Noti

Ci troviamo davanti ad un codice assembly, ovvero un linguaggio di basso livello. Con linguaggi di basso livello si intendono un sottogruppo dei linguaggi di programmazione orientanti alla macchina, piuttosto del linguaggio di programmazione ad alto livello che è orientato all'utente (come il C che è di livello alto, o python che è di tipo interpretativo)

In questa prima parte facciamo un'analisi del codice fornito per identificare i costrutti noti.

```
push ebp
mov ebp, esp
```

Questo blocco ha il compito di stabilire un nuovo frame dello stack per la funzione corrente.

```
push ecx
push 0
push 0
call ds:InternetGetConnectedState
```

Questo blocco di istruzioni **chiama la funzione InternetGetConnectedState** per determinare lo stato della connessione Internet. Le istruzioni push sono utilizzate per passare i parametri necessari alla funzione, mentre la call effettua la chiamata vera e propria.

```
mov [ebp+var_4], eax
```

Inizializzazione di una variabile locale tramite il valore del registro EAX (che contiene il ritorno della funzione InternetGetConnectedState)

```
cmp [ebp+var_4], 0
jz Short loc_40102B
```

Qui possiamo riconoscere il costrutto **IF**, dove il codice controlla il risultato della funzione InternetGetConnectedState. Se l'operazione **NON** è andata a buon fine, il programma passa alla sezione di codice che si trova in posizione 40102B.

Nota: Da qui abbiamo una biforcazione del codice che dipende dal IF soprastante. Analizziamo prima il caso in cui il malware è riuscito a stabilire una connessione a Internet.

```
push offset aSuccessInterne ; "Success: Internet Connection\n"
```

```
call sub_40105F
```

Queste istruzioni rappresentano una chiamata a una funzione in cui viene passato un parametro, specificamente una stringa. Purtroppo, la parte di codice a cui fa riferimento la *call* non è a nostra disposizione e quindi non sappiamo cosa va a fare.

```
add esp, 4
```

Questa istruzione serve per pulire lo stack.

```
mov ecx, 1
```

Imposta il registro **ecx** a 1, forse in questo caso viene usato per restituire un valore o come codice di stato.

```
jmp Short loc_40103A
```

Viene effettuato un salto incondizionato verso l'indirizzo specificato.

Nota: Ora analizziamo il caso in cui **NON** è stato possibile stabilire una connessione

```
push offset aError1_1NoInte
```

```
call sub_40117F
```

La sequenza di istruzioni *push offset aError1_1NoInte* e *call sub_40117F* mostra il passaggio di una stringa contenente un messaggio di errore che, probabilmente, viene passato alla funzione presente all'indirizzo *sub_40117F* (di cui non disponiamo il codice) per gestire l'errore di connessione.

```
add esp, 4
```

```
xor eax, eax
```

Le istruzioni *add esp, 4* e *xor eax, eax* sono utilizzate per ripulire lo stack dopo una chiamata di funzione e per impostare un registro a zero.

Nota: Indipendentemente dalla condizione dell'IF viene effettuato quest'ultima parte di codice.

```
mov esp, ebp
```

```
pop ebp
```

```
retn
```

```
sub_401000 endp
```

Queste istruzioni sono usate per ripristinare lo stack frame e restituire il controllo alla funzione chiamante.

4) Ipotesi funzionamento

Questo codice assembly sembra essere parte di una funzione che verifica lo stato della connessione Internet e, in base al risultato, esegue delle azioni specifiche.

- **Verifica dello Stato della Connessione Internet:**
 - **Funzione Utilizzata:** La funzione utilizza `InternetGetConnectedState` per verificare se il computer è attualmente connesso a Internet.
 - **Controllo del Risultato:** Dopo la chiamata a `InternetGetConnectedState`, il risultato viene confrontato utilizzando le istruzioni `cmp` e `jz` (jump if zero). Se il risultato indica che non c'è connessione Internet, il flusso del programma salta a una sezione di gestione dell'errore.
- **Gestione del Successo:**
 - **Azioni in Caso di Connessione Attiva:** Se `InternetGetConnectedState` indica che c'è una connessione Internet, la funzione chiama `sub_40105F`, presumibilmente per stampare un messaggio di successo ("Success: Internet Connection\n"). Inoltre, il registro `ecx` viene impostato a 1, probabilmente per indicare un esito positivo della funzione.
- **Gestione dell'Errore:**
 - **Azioni in Caso di Mancanza di Connessione:** Se non c'è una connessione Internet, la funzione chiama `sub_40117F` per stampare un messaggio di errore ("Error: Internet Connection Failed"). Inoltre, il registro `eax` viene azzerato (impostato a zero), suggerendo che la funzione restituisce un valore di errore.
- **Pulizia e Restituzione:**
 - **Ripristino del Contesto:** Alla fine della funzione, il contesto dello stack viene ripristinato e i registri di base vengono riportati ai loro valori precedenti. Infine, il controllo viene restituito alla funzione chiamante tramite l'istruzione `retn`.

5) Descrizione codice

Codice	Descrizione
push ebp	Salva il valore corrente di EBP (Extended Base Pointer) sullo stack
mov ebp, esp	Imposta EBP al valore corrente di ESP (Extended Stack Pointer) per creare un nuovo stack frame
push ecx	Salva il valore corrente del registro ECX sullo stack
push 0	Spinge il valore 0 sullo stack come secondo parametro per InternetGetConnectedState
push 0	Spinge un altro valore 0 sullo stack come primo parametro per InternetGetConnectedState.
call ds: InternetGetConnectedState	Chiama la funzione InternetGetConnectedState che determina lo stato della connessione Internet. Il risultato della funzione viene restituito nel registro EAX. Se la funzione riesce a connettersi restituisce TRUE (1) altrimenti restituisce FALSE (0) . La funzione è contenuta nella libreria wininet.dll (che abbiamo trovato nella analisi statica).
mov [ebp+var_4], eax	Salva il risultato di InternetGetConnectedState in una variabile locale (var_4)
cmp [ebp+var_4], 0	Confronta il risultato salvato con 0
jz Short loc_40102B	Se il risultato è zero (cioè, non c'è connessione Internet), salta all'etichetta loc_40102B
push offset aSuccessInterne	Spinge l'indirizzo della stringa "Success: Internet Connection\n" sullo stack
call sub_40105F	Chiama una subroutine (probabilmente per stampare il messaggio di successo)
add esp, 4	Rimuove il parametro dalla stack
mov ecx, 1	Imposta ECX a 1 per indicare successo
jmp Short loc_40103A	Salta all'etichetta loc_40103A, bypassando il codice di errore
loc_40102B	Etichetta raggiunta se non c'è connessione Internet
push offset aError1_1NoInte	Spinge l'indirizzo della stringa "Error: Internet Connection Failed" sullo stack
call sub_40117F	Chiama una subroutine (probabilmente per stampare il messaggio di errore)
add esp, 4	Rimuove il parametro dalla stack
xor eax, eax	Imposta EAX a zero per indicare un errore
loc_40103A	Etichetta raggiunta se c'è una connessione a Internet
mov esp, ebp	Ripristina ESP al valore originale salvato in EBP
pop ebp	Ripristina EBP al valore originale salvato all'inizio della funzione
ret	Restituisce il controllo alla funzione chiamante

BONUS

Un giovane dipendente neoassunto segnala al reparto tecnico la presenza di un programma sospetto. Il suo superiore gli dice di stare tranquillo ma lui non è soddisfatto e chiede supporto al SOC. Il file "sospetto" è iexplore.exe contenuto nella cartella C:\Programmi\Internet Explorer (no, non ridete ragazzi)

Come membro senior del SOC ti è richiesto di convincere il dipendente che il file non è maligno. Possono essere usati gli strumenti di analisi statica basica e/o analisi dinamica basica visti a lezione.

No disassembly no debug o similari VirusTotal non basta, ovviamente Non basta dire iexplorer è Microsoft quindi è buono, punto.

Analisi del Software

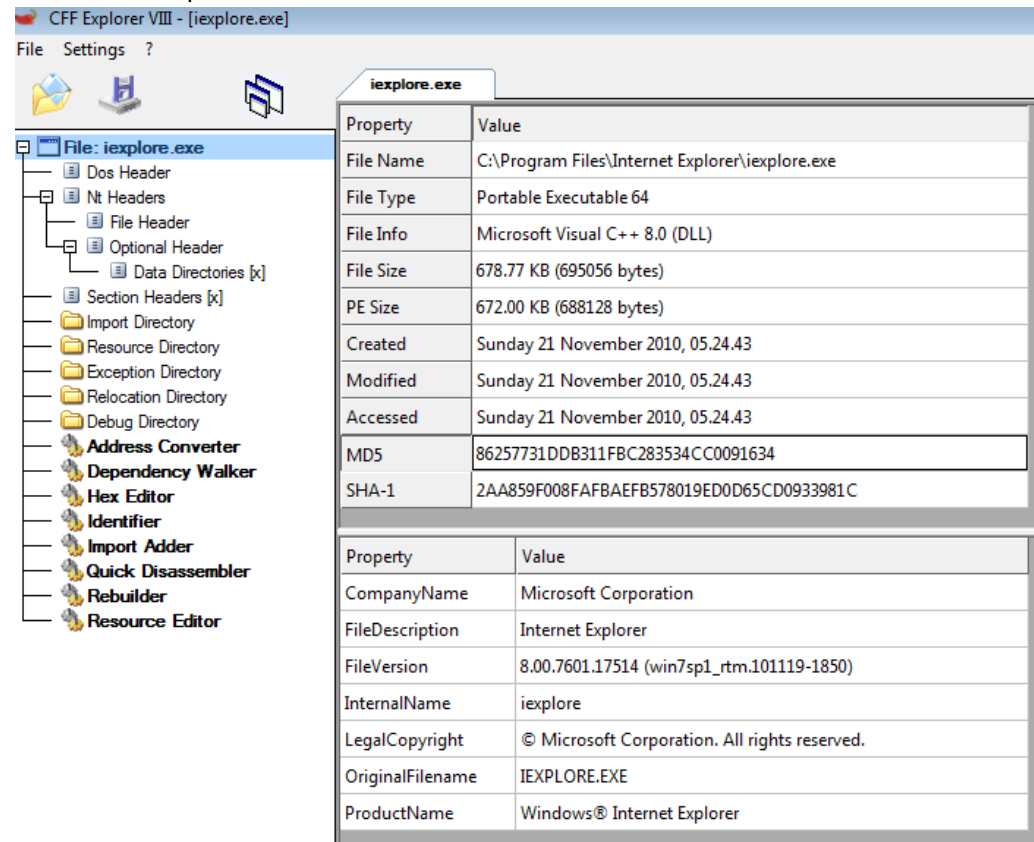
Per accertarci che il software Internet Explorer non sia un file malevolo possiamo eseguire un'analisi statica e un'analisi dinamica.

Analisi Statica

Carichiamo sul programma CFF Explorer il file eseguibile iexplorer.exe e cerchiamo indizi sul potenziale malevolo del software.

1. Controllo Proprietario

Nella prima schermata di CFF Explorer possiamo vedere alcune informazioni importanti, come il proprietario del Software che in questo caso è Microsoft Corporation.



CFF Explorer VIII - [iexplorer.exe]

File Settings ?

File: iexplorer.exe

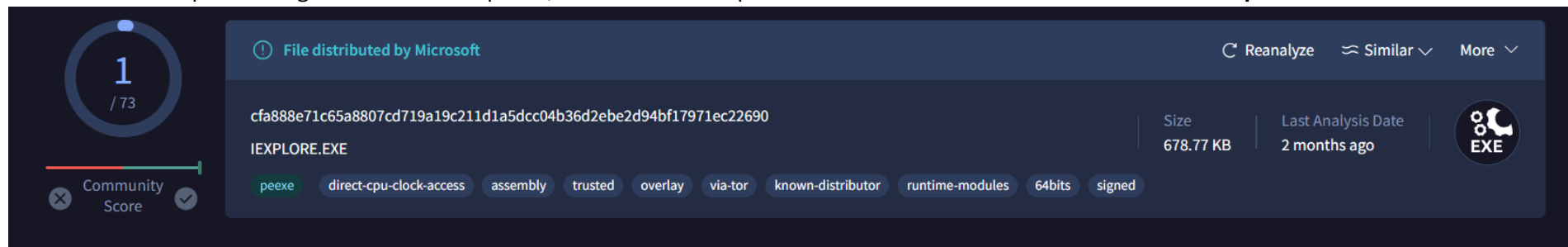
- Dos Header
- Nt Headers
 - File Header
 - Optional Header
 - Data Directories [x]
- Section Headers [x]
- Import Directory
- Resource Directory
- Exception Directory
- Relocation Directory
- Debug Directory
- Address Converter
- Dependency Walker
- Hex Editor
- Identifier
- Import Adder
- Quick Disassembler
- Rebuilder
- Resource Editor

Property	Value
File Name	C:\Program Files\Internet Explorer\iexplorer.exe
File Type	Portable Executable 64
File Info	Microsoft Visual C++ 8.0 (DLL)
File Size	678.77 KB (695056 bytes)
PE Size	672.00 KB (688128 bytes)
Created	Sunday 21 November 2010, 05.24.43
Modified	Sunday 21 November 2010, 05.24.43
Accessed	Sunday 21 November 2010, 05.24.43
MD5	86257731DDB311FBC283534CC0091634
SHA-1	2AA859F008FAFBAEFB578019ED0D65CD0933981C

Property	Value
CompanyName	Microsoft Corporation
FileDescription	Internet Explorer
FileVersion	8.00.7601.17514 (win7sp1_rtm.101119-1850)
InternalName	iexplore
LegalCopyright	© Microsoft Corporation. All rights reserved.
OriginalFilename	IEXPLORE.EXE
ProductName	Windows® Internet Explorer

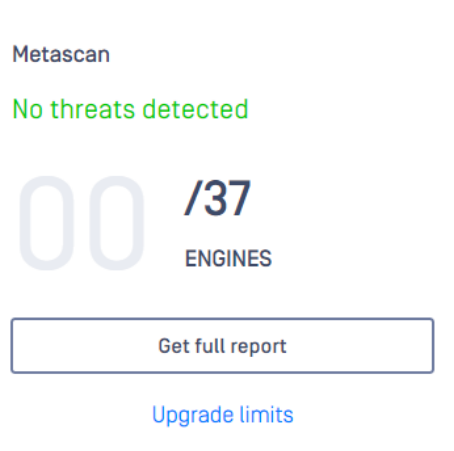
2. Analisi hash

Un altro step per controllare l'autenticità del software è quello di controllare l'hash su varie piattaforme, come virustotal ad esempio. L'hash ci viene messo a disposizione già dal tool CFF Explorer, ma in alternativa possiamo calcolarlo con altri tool come **md5deep**.



VirusTotal ci mostra su 73 Security vendors' analysis soltanto 1 sembra identificarlo come potenzialmente malevolo.

Invece, Metadefender ci mostra zero segnalazioni contro iexplore.exe.



3. Verifica delle librerie importate

Possiamo analizzare quali librerie vengono caricate dal software.

iexplore.exe						
Module Name	Imports	OFTs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
ADVAPI32.dll	13	0000F6B8	FFFFFFFF	FFFFFFFF	0000F6A8	00009000
KERNEL32.dll	56	0000F728	FFFFFFFF	FFFFFFFF	0000F698	00009070
USER32.dll	9	0000F8F0	FFFFFFFF	FFFFFFFF	0000F68C	00009238
msvcrt.dll	29	0000F940	FFFFFFFF	FFFFFFFF	0000F680	00009288
ntdll.dll	3	0000FA30	FFFFFFFF	FFFFFFFF	0000F674	00009378
SHLWAPI.dll	23	0000FA50	FFFFFFFF	FFFFFFFF	0000F668	00009398
SHELL32.dll	7	0000FB10	FFFFFFFF	FFFFFFFF	0000F65C	00009458
ole32.dll	5	0000FB50	FFFFFFFF	FFFFFFFF	0000F650	00009498
iertutil.dll	14	0000FB80	FFFFFFFF	FFFFFFFF	0000F640	000094C8
urlmon.dll	3	0000FBF8	FFFFFFFF	FFFFFFFF	0000F634	00009540

Vediamo che vengono mostrare diverse librerie in “chiaro”. Questo comportamento è piuttosto atipico per un malware visto che, in genere, cercano di limitare il più possibile il numero di librerie mostrate, così da nascondere più efficacemente le proprie funzionalità. Infatti, i malware preferiscono un approccio **run time** per l’importazione delle librerie usando funzioni come **GetProcAddress** e **LoadLibrary**. Queste funzioni vengono usate anche da iexplore.exe, ma, come detto precedentemente, il software in questione usa diverse librerie in chiaro.

4. Analisi delle sezioni

Alcuni malware nascondono i nomi delle proprie sezioni o ne hanno alcune aggiuntive, magari inesistenti, rispetto alle tradizionali. Tramite CFF Explorer possiamo vedere numero e nome delle varie sezioni che compongono il software.

iexplore.exe									
Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00007349	00001000	00007400	00000400	00000000	00000000	0000	0000	60000020
.rdata	00007C08	00009000	00007E00	00007800	00000000	00000000	0000	0000	40000040
.data	00000B0C	00011000	00000A00	0000F600	00000000	00000000	0000	0000	C0000040
.pdata	00000564	00012000	00000600	00010000	00000000	00000000	0000	0000	40000040
.rsrc	00097020	00013000	00097200	00010600	00000000	00000000	0000	0000	40000040
.reloc	00000674	000AB000	00000800	000A7800	00000000	00000000	0000	0000	42000040

In questo caso, i nomi delle sezioni sono leggibili. Notiamo che, rispetto a casi visti in precedenza, abbiamo alcune sezioni in più come **.pdata** e **.reloc**. Indagando su queste sezioni scopriamo che la sezione **.pdata** supporta la gestione delle eccezioni, mentre **.reloc** fornisce informazioni di rilocalizzazione necessarie per aggiornare gli indirizzi quando il modulo è caricato in memoria. Non ci sono sezioni ambigue o fuori dal comune.

5. Conclusione Analisi Statica

In questa analisi statica non abbiamo trovato nulla che possa far sospettare che il software sia malevolo. Infatti, abbiamo trovato un proprietario conosciuto e affidabile (Microsoft); tramite hash abbiamo fatto un confronto con i database messi a disposizione da VirusTotal e MetaDefender trovando una sola segnalazione sospetta; le librerie importanti e il fatto che non siano nascoste si scontrano con il classico comportamento di un malware; le sezioni sono comuni e legittime. In base a tutti questi controlli il software può definirsi sicuro.

Analisi Dinamica

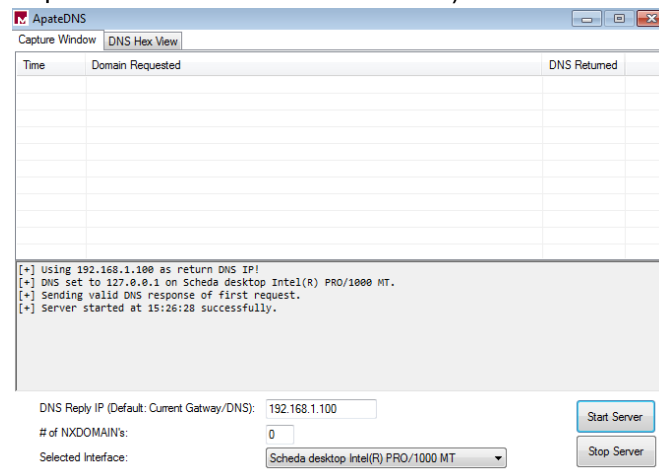
Per effettuare un'analisi dinamica basica abbiamo bisogno di controllare i processi avviati, le modifiche ai registri, modifiche alle file system e tentativi di connessioni verso internet (collegamenti a siti malevoli).

I tool che andremo ad usare sono:

- **Process Explorer** per i processi;
- **ApateDNS** per controllare le richieste DNS;
- **Procmon** per analizzare varie attività come le modifiche al file system;
- **Regshot** per osservare eventuali modifiche ai registri.

1. Preparazione dell'ambiente e dei tool

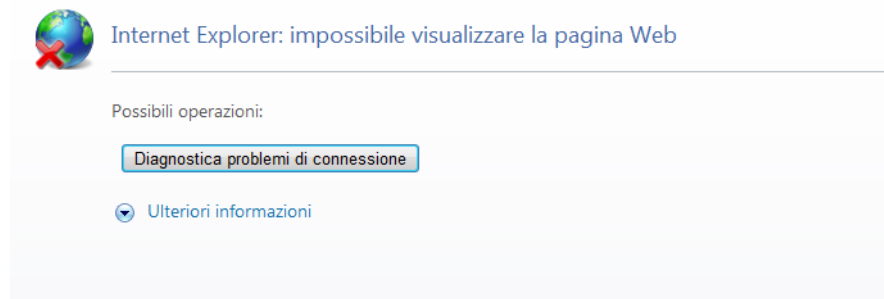
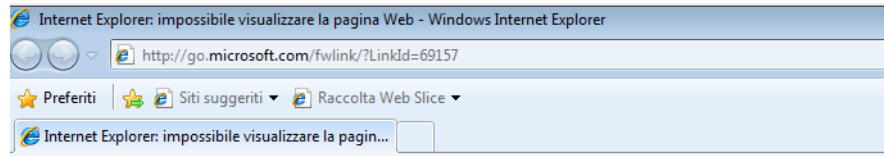
- Per evitare la propagazione del potenziale malware creiamo una macchina virtuale isolata dall'accesso a internet, periferiche esterne (come USB) e dall'host. Inoltre, creiamo una istantanea della macchina in caso sia necessario un ripristino.
- Avviamo **Process Explorer**.
- Configuriamo il server DNS con **ApateDNS** impostando l'IP con l'indirizzo IP 192.168.1.100 (configurazione da apportare anche nelle impostazioni della rete di Windows)



- Usiamo **Regshot** per salvare una prima istantanea dello stato delle chiavi di registro
- Avviamo **Procmon** e **Wireshark**

2. Esecuzione del software

- Avviamo iexplore.exe ed osserviamone il comportamento per un certo lasso di tempo.
- La prima interazione visibile che abbiamo con il software è l'apertura del browser Internet Explorer. Essendo la macchina isolata, il software non riesce a raggiungere la pagina di default.



- Tramite Process Explorer vediamo che è stato creato un processo iexplore.exe. Non sembrano esserci altri processi creati dal programma.

iexplore.exe		10.368 K	27.904 K	572 Internet Explorer	Microsoft Corporation
	0.06	10.132 K	28.224 K	1068 Internet Explorer	Microsoft Corporation

- Chiudiamo il processo e passiamo alla fase di analisi.

3. Analisi post-esecuzione

- **Analisi Regshot**

Eseguiamo una seconda istantanea con Regshot e la confrontiamo con la prima.

Vediamo che sono state aggiunte due nuove chiavi dal processo iexplore.exe

keys added: 29

HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS

Queste chiavi sono utilizzate dal sistema operativo per registrare eventi e diagnosticare problemi relativi alla connessione di rete e ai servizi di rete.

All'interno di queste chiavi sono state modificate le seguenti voci di registro.

values added: 57

HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\EnableFileTracing: 0x00000000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\EnableConsoleTracing: 0x00000000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\FileTracingMask: 0xFFFF0000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\ConsoleTracingMask: 0xFFFF0000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\MaxFileSize: 0x00100000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASAPI32\FileDirectory: "%windir%\tracing"
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\EnableFileTracing: 0x00000000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\EnableConsoleTracing: 0x00000000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\FileTracingMask: 0xFFFF0000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\ConsoleTracingMask: 0xFFFF0000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\MaxFileSize: 0x00100000
HKLM\SOFTWARE\Microsoft\Tracing\iexplore_RASMANCS\FileDirectory: "%windir%\tracing"

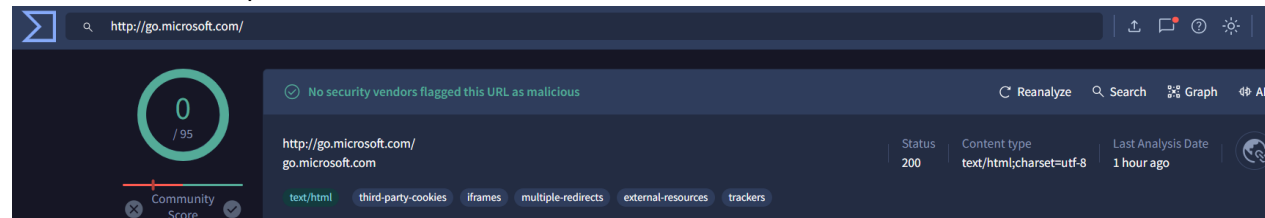
Le voci del registro di sistema non sembrano indicare nulla di sospetto.

- **Analisi ApateDNS**

ApateDNS mostra un tentativo di connessione verso il sito **go.microsoft.com**

[illegible]

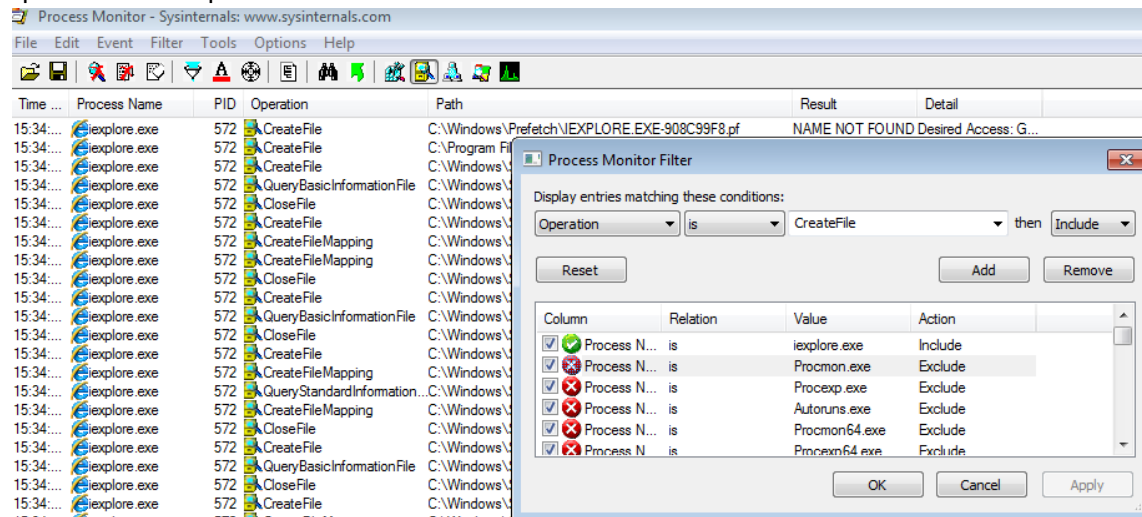
Usiamo VirusTotal per analizzare se l'URL è sicuro o meno



Nessuno dei 95 Security vendors' analysis indica questo sito come malevolo.

- **Analisi file system tramite Procmon**

Essendoci un elevato numero di eventi dobbiamo applicare dei filtri. Innanzitutto, usiamo un filtro basato sul nome del processo, in questo caso iexplore.exe



Successivamente possiamo applicare altri filtri per ridurre ulteriormente il campo di analisi, come Operation o anche Path che possono essere sensibili. Visto l'elevato numero di interazioni, l'unica cosa che è stata possibile fare è un'analisi a campione che non ha rilevato nulla di anomalo.

4. **Conclusione Analisi Dinamica**

Anche con l'analisi dinamica base non abbiamo trovato nulla di sospetto. Registri, processi e tentativi di connessioni a siti esterni non sembrano essere né fuori dalla norma, né malevoli. Al contrario alcuni tool usati hanno sottolineato un certo grado di affidabilità del software, come ad esempio VirusTotal. Per questo, possiamo affermare che l'exe iexplore è da considerare affidabile.